CSE 493S Empirical Methods Assignment

Progress Report: Finished Deliverables

Aadyant Maity and Rahul Bonthu

May 1, 2026

Part 0: implementation

Part 0 is basically the codebase the rest of the assignment builds on. Everything lives under `deliverables/part_0/`: the usual scripts (`train.py`, `model.py`, `inference.py`, `tokenizer_utils.py`, `arith_data.py`) plus `part_0_1_contract.py` for the autograder-facing API.

We sanity-check with the quick configs under `deliverables/part_0/configs/`, for example:

```
python train.py --config configs/sanity_quick.yaml
python train.py --config configs/sanity_suffix_quick.yaml
```

(run from the `EmpiricalHW/` root, or point paths appropriately). Nothing fancy to report here besides "it trains and the contract loads checkpoints"—the interesting experiments start in Part 1.

Part 1.1: modular arithmetic datasets

For Part 1.1 we generated text files of equations `a op b = c` with arithmetic mod p. Addition and subtraction iterate all pairs (a, b) with $0 \le a, b < p$. Division skips $b = 0$ and uses $b \in \{1, \ldots, p-1\}$, so division sets are a bit smaller than add/sub for the same p.

We shuffled once with seed 0, then split each file **80% / 10% / 10%** train, validation, and test in that order. Every `meta.json` matches the lines in the matching `.txt` files.

Split sizes

These counts match what is on disk under `deliverables/part_1/1.1/data/`.

Table 1: Number of lines per split for each operator and modulus.

p	op	train	val	test
97	`+ / -`	7527	940	942
97	`/`	7449	931	932
113	`+ / -`	10215	1276	1278
113	`/`	10124	1265	1267

To regenerate from the course workflow:

```
python arith_data.py --seed 0
```

Part 1.2: addition mod 97 (one-layer, deliverable run)

Part 1.2 is the addition-mod-97 setup with a one-layer model. Our submitted bundle lives under `deliverables/part_1/1.2/checkpoints/p97_add_1L_restart1_seed101/`, with final weights `ckpt_100000.pt` (`max_steps` = 100000, `save_every` = 10000), plus `tokenizer.json`, `metrics.csv`, and resolved config metadata.

Training curves (seed 101)

Figure 1 shows train/val/test loss, token accuracy, and equation accuracy for steps 0–100000 only (we pass `--max-step 100000` to `plot_metrics.py` so the figure matches the deliverable budget exactly).

Numbers at the last logged step (seed 101)

At step 100000 (last row of the deliverable `metrics.csv`), we see roughly:

- Train loss ≈ 0.0136, train token acc ≈ 0.996.
- Val loss ≈ 0.0218, val token acc ≈ 0.993.
- Test loss ≈ 0.0275, test token acc ≈ 0.993.
- Train / val / test equation acc ≈ 0.985, 0.974, and 0.972.

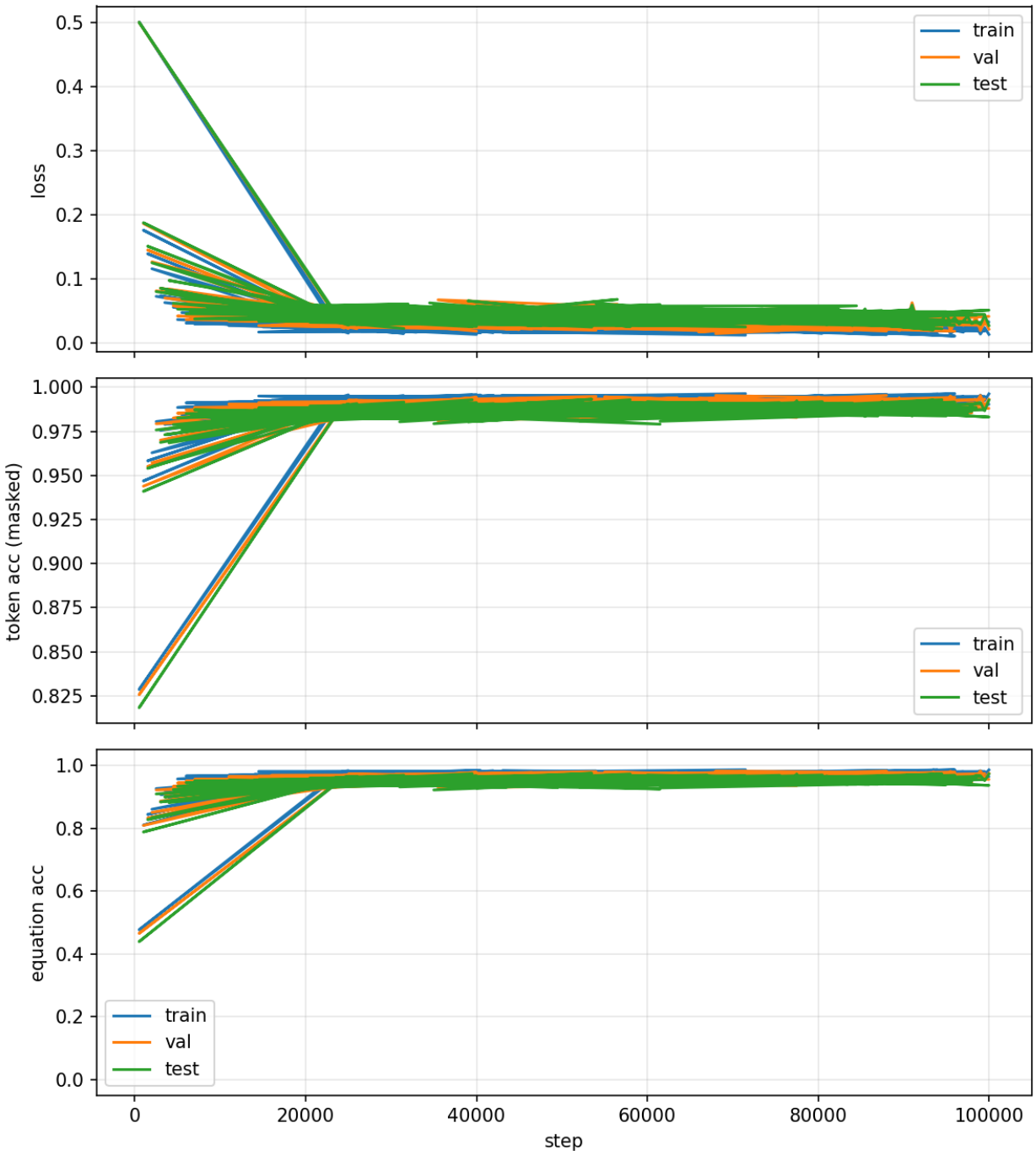

Figure 1: Part 1.2 deliverable run: $p = 97$, addition, one layer, seed 101, 100k steps.

Part 1.3: division mod 97 (grokking-style run)

Part 1.3 trains on division modulo 97. The deliverable bundle is under `deliverables/part_1/1.3/checkpoints/part1_3_div_p97/`, with final weights `ckpt_300000.pt`, `tokenizer.json`, `metrics.csv`, and resolved config metadata.

Training curves

Figure 2 shows loss and accuracy vs. step from `metrics.csv`. We generated the PNG with `plot_metrics.py` from that run's CSV (repo-root script, `--max-step` optional).

Numbers at the last logged step

At step 300000 (last row of `metrics.csv`), the CSV reports roughly:

- Train loss ≈ 0.331, train token acc ≈ 0.888.
- Val loss ≈ 2.38, val token acc ≈ 0.560.
- Test loss ≈ 2.31, test token acc ≈ 0.566.
- Train equation acc ≈ 0.618; val/test equation acc stayed low on the last rows (this is consistent with the grokking story where memorization on train can run ahead of generalization on full equations).

We are not over-interpreting these here—the figure is the main takeaway for us until we write up discussion elsewhere.

How to run inference

From `deliverables/part_1/1.3/code/`, we can use either free-form generation or `predict_answer` on integers. Example:

```
python inference.py --checkpoint_dir ../checkpoints/part1_3_div_p97 \
    --prompt "3 / 5 = "
```

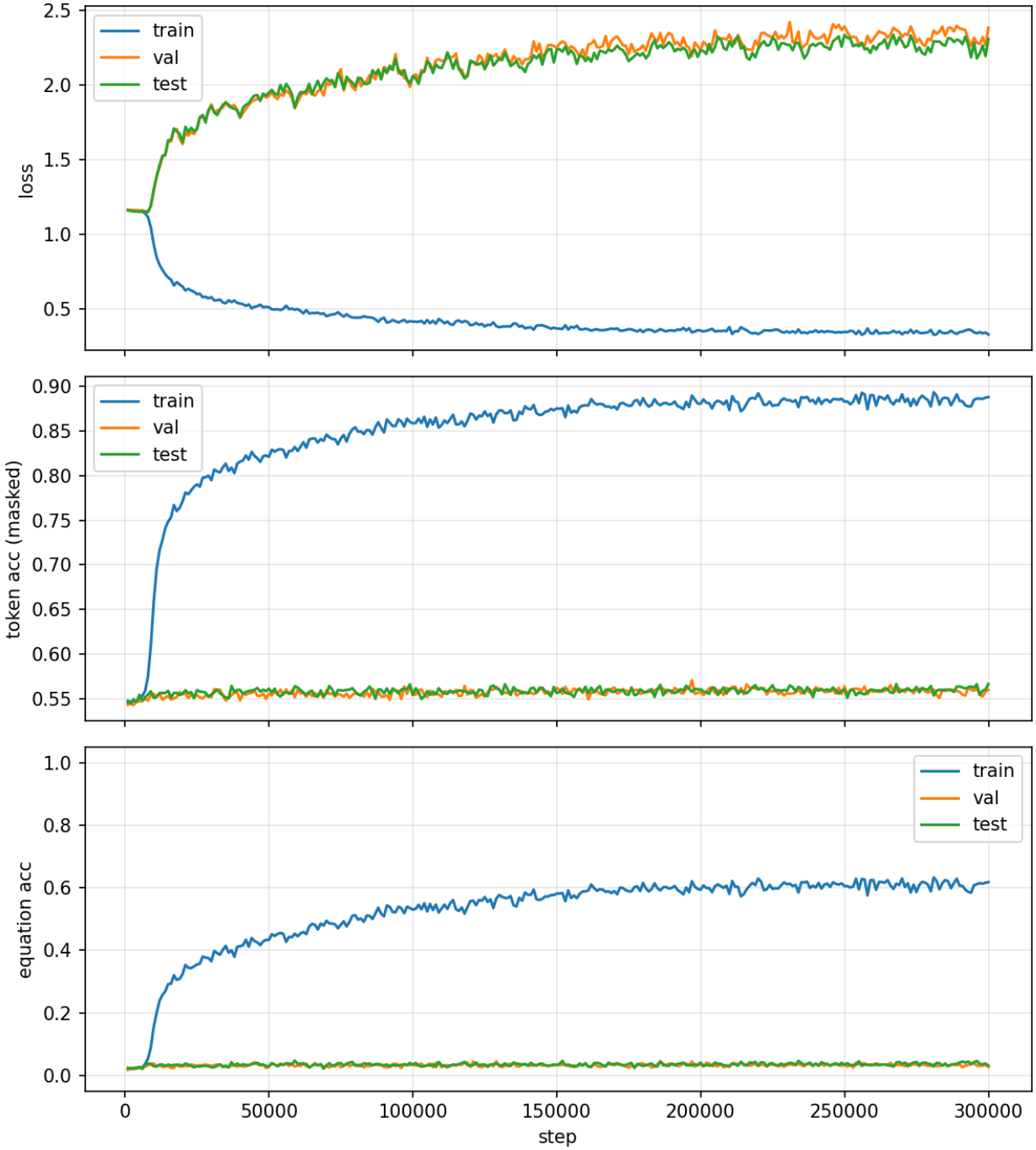

Figure 2: Training / validation / test curves for the division mod 97 experiment (`part1_3_div_p97`).

Part 1.4: ablations (division mod 97)

Part 1.4 compares two tweaks to the Part 1.3-style division setup ($p = 97$, one layer, 300k steps, seed 42). Each ablation has its own checkpoint folder under `deliverables/part_1/1.4/checkpoints/`.

Dropout (`dropout` $= 0.2$, `weight_decay` $= 1.0$)

Bundle: `part_1/1.4/checkpoints/part1_4_ablation_dropout/`.
Figure 3: `plot_metrics.py` with `--max-step 300000`.

At step 300000 (last row of that run's `metrics.csv`):

- Train loss ≈ 1.16, train token acc ≈ 0.546.
- Val loss ≈ 1.16, val token acc ≈ 0.540.
- Test loss ≈ 1.16, test token acc ≈ 0.548.
- Train / val / test equation acc ≈ 0.021, 0.016, and 0.021.

Low weight decay (`weight_decay` $= 0.1$, `dropout` $= 0.0$)

Bundle: `part_1/1.4/checkpoints/part1_4_ablation_wd_low/`.
Figure 4: same `plot_metrics.py` setup with that run's `metrics.csv`.

At step 300000 (last row of that run's `metrics.csv`):

- Train loss ≈ 0.065, train token acc ≈ 0.979.
- Val loss ≈ 5.63, val token acc ≈ 0.546.
- Test loss ≈ 5.58, test token acc ≈ 0.541.
- Train equation acc ≈ 0.921; val / test equation acc ≈ 0.025 and 0.017 (strong train fit, poor held-out equation generalization at this step budget).

Figure 3: Part 1.4 dropout ablation: division mod 97, `dropout` $= 0.2$, 300k steps.

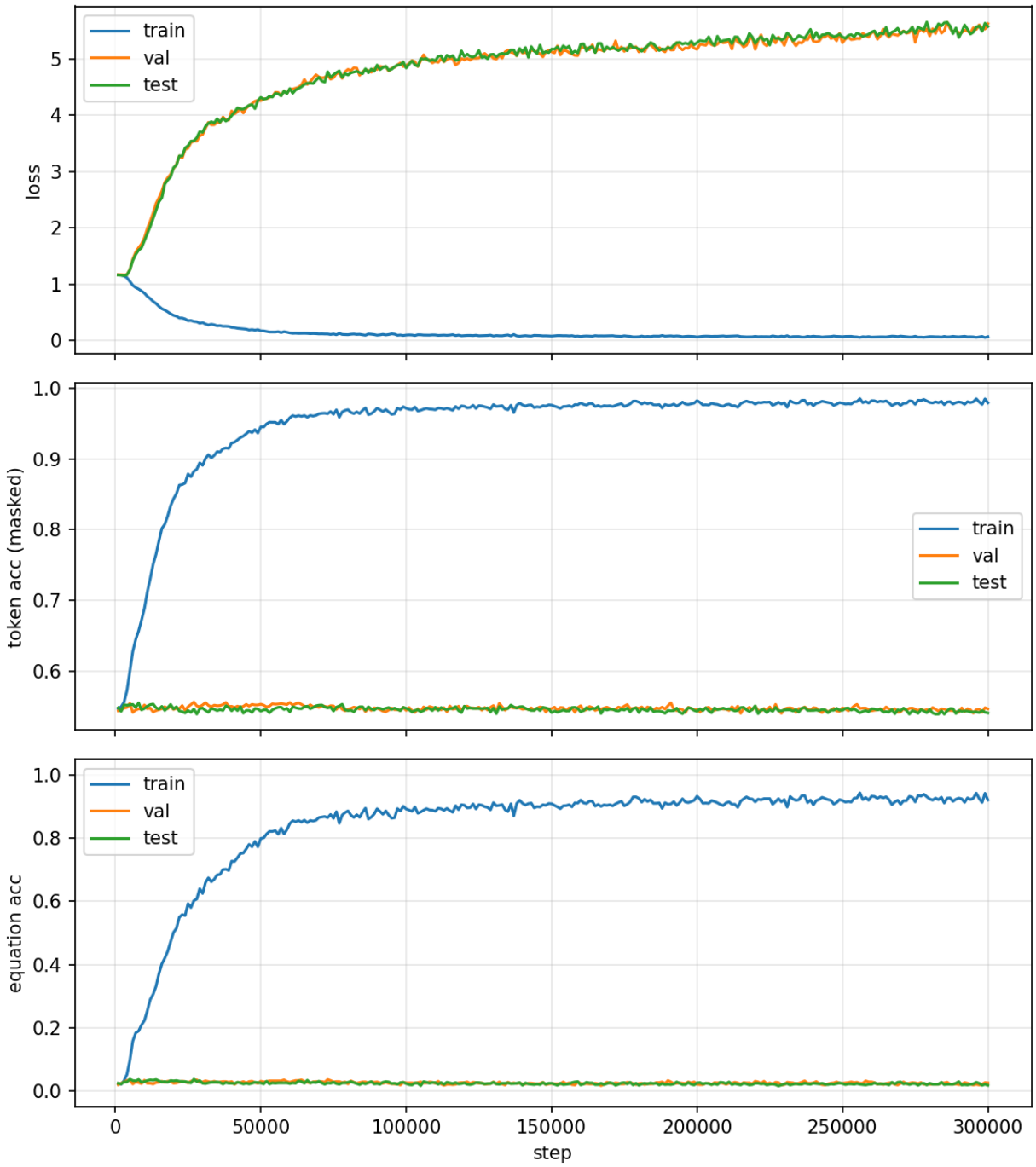

Figure 4: Part 1.4 low weight decay ablation: division mod 97, `weight_decay` = 0.1, 300k steps.

Brief analysis

We reused the Part 1.3 division setup (mod 97, one layer, 300k steps, seed 42) and only changed what the homework suggested we play with: one run adds `dropout` $= 0.2$, the other drops `weight_decay` to 0.1 and turns dropout off (`deliverables/part_1/1.4/configs/`). Motivation-wise we were not trying to be clever—dropout is just "more regularization noise," and lower WD is the knob people always mention when grokking shows up or does not, so we wanted to see if either run looked qualitatively closer to a delayed test-time jump on the plots.

The dropout curves basically all move together and never really split into a "train figured it out, test still dying" story; it mostly looks stuck. Numbers-wise loss sits around 1.16, token acc stays near 0.54–0.55, and equation acc ends near $\sim 2\%$ on train/val/test, so nothing in the figure screams "about to generalize soon."

Low WD is the opposite vibe: training gets almost easy—train loss ≈ 0.065, train equation acc ≈ 0.92—but val/test loss blow up (~ 5.6) and val/test equation acc stay at a few percent ($\approx 2.5\%$ / 1.7%). To us that reads like memorizing the train split without the homework-style payoff where test equation accuracy suddenly catches up, at least not before step 300000.

The prompt also asks for a step count where test error hits zero after train already looks solved. We never saw test equation accuracy near 1 in either ablation (same rough situation as our Part 1.3 checkpoint), so we cannot honestly quote a "grokking delay" in steps—only that the two tweaks change whether the failure mode looks like "nothing learns" versus "train looks great, test still random-ish on full equations."